



Python in Research, Part 4

Optimising Returns, Minimising Risk

Can SciPy be of use in economics and social sciences? The answer, of course, is, "Yes!"

One of the best-known methods to minimise risk is insurance. However, it came into existence in the 17th century.

Economic theory, like insurance, is heavily based on statistics. Economic models are not perfect, as the global recession of the last few years has shown. It is not that the modelling is wrong. Rather, more complex models are needed. The role that human behaviour plays needs to be factored in.

As models get more complex, spreadsheets with macros will not be adequate and we'll require more complex tools. An example of the possibilities is an interesting commercial product called Resolver. This blends a spreadsheet-like interface with the ability to program in Python (IronPython) and includes some support for NumPy.

It is likely that SciPy will find proponents among social scientists as well. You can be among the early converts. Explore a few trivial examples in order to get an idea of the tools available.

Reducing risk by diversification

The risk for an investment can be estimated by the standard deviation of the result. Consider the following options:

- Investment 1 at 10% return with a standard deviation of 10%

- Investment 2 at 15% return with a standard deviation of 20%

Suppose a fraction p is invested in Investment 1 and the remaining $(1-p)$ in Investment 2. What would be the resulting standard deviation? The textbook formula is:

$$\sigma^2 = p^2 \sigma_1^2 + (1-p)^2 \sigma_2^2 + 2p(1-p) \rho_{12} \sigma_1 \sigma_2$$

where σ_i is the standard deviation of the i^{th} investment

ρ_{12} is the correlation between the two investments

You can use matplotlib [matplotlib.sourceforge.net] to plot this equation for several values of correlation and visualise it. Figure 1 shows the plot.

```
import scipy as np
def std_dev(p, c12):
    vpsq = (p*v1)**2 + ((1-p)*v2)**2 + p*(1-p)*c12*v1*v2
    return np.sqrt(vpsq)
def avg_return(p):
    return p*r1 + (1-p)*r2
r1, v1 = 10, 10
r2, v2 = 15, 20
p = np.arange(0.1, 0.9)
# Plot the results
import matplotlib.pyplot as plt
```